Written Problem And Solution Statement

Problem

Growing companies spend money through many disconnected systems: supplier invoices, credit cards, bank transfers, cloud bills, SaaS subscriptions, payroll, and more. No one person sees all of it, so waste survives in the gaps between systems.

Finance teams usually look for these leaks in monthly spreadsheets and sampled annual audits. Specialist tools help, but most cover a single category, such as cloud or SaaS. A CFO at a 40 to 500 person company therefore has no evidence-backed answer to a basic question: what are we paying for that we don't need, and what should we do about it? For a startup, that lost money is lost runway.

Scenario

A finance lead at a 91-person tech company is closing the month. Each system looks fine on its own, but together they hide costly mistakes. The same AWS invoice was paid twice across two currencies (₪44,400). A former employee was still paid after their employment ended (₪72,000). A staging server keeps billing while doing no work (₪51,969). Eight of 14 Figma licences are unused (₪13,440). These examples come from running our engine on a synthetic company we built for testing.

Solution

Where Is the Money (איפה הכסף) scans a company's financial and usage data and returns a list of findings, each with an amount at risk, a confidence score, the original rows as evidence, and one recommended action. Deterministic rules written in code make every decision, so each amount can be traced to its source. No evidence means no finding. A finding is a prompt to review a process, never a judgment of a person.

Our working proof of concept runs 10 rules covering duplicate payments, payroll anomalies, unused or duplicate licences, price jumps at renewal, and idle cloud resources. On the synthetic dataset it produces 34 findings totalling ₪2.58M flagged for review. This is not all savings: ₪1.64M of it is invoice-splitting patterns that need a human decision. Results appear in a Hebrew dashboard.

Next, we will expand to the 33 rules already specified, including AI spend and Israeli tax details, add read-only connectors, and add a conversational agent on watsonx Orchestrate. We have prepared a 155,263-row synthetic test dataset in real export formats, with 26 planted issues, to measure that work. The proposed business model is a free first scan followed by a subscription, which we will validate with pilot customers.

Written Statement On Technology

IBM Bob: We used Bob IDE with four custom modes (UI designer, rules engineer, product manager, pitch architect), project rules for output format and privacy, and an MCP configuration. Only the filesystem connector is enabled; the others are prepared but disabled. Our Bob task session screenshots are in the bob_sessions folder. Parts of the work, including the v2 specification, synthetic data, design system and CI/CD, were developed with Claude Code, as the git history shows.

watsonx Orchestrate: The current proof of concept does not use watsonx Orchestrate. We have specified an agent that will answer finance questions in Hebrew using only our findings data and will cite the source rows. Any action, such as messaging a vendor, will require human approval. This agent and the MCP server it relies on are not yet built.

Rules engine: Python and pandas. Each rule is a separate module that returns the same Finding structure, with 26 passing pytest unit tests. The engine does not use an LLM to decide what is anomalous or to calculate amounts.

API: A FastAPI layer is in progress, with endpoints designed to scan CSV files, list findings, update a finding's status, and read summary statistics. It is not yet deployed, and the synthetic-data scan endpoint currently has a known bug.

Dashboard: HTML and vanilla JavaScript, right-to-left in Hebrew. The deployed version reads an exported findings file. The API is not yet deployed.

Data: All company data is synthetic. No real employee data is used, and API keys are kept out of the code.

Delivery: GitHub Actions runs secret scanning (gitleaks), policy checks, linting (ruff), rule tests, synthetic data validation, browser smoke tests (Playwright), and a Docker image check. The site is served by nginx in Docker at eifo-money.fit.

Design prototype: The screens at eifo-money.fit/design/v2/ show the target product using demo data and are not connected to the engine.

Code Repository

https://github.com/yossiheinebiz-cloud/eifo-money

Links (Optional)

Live dashboard: https://eifo-money.fit/dashboard/